

Multi-Agent LLM Research Automation Platform: Architecture, Implementation, and System Evaluation

Om Choksi

Devang Dhandhukiya

Department of Artificial Intelligence and Machine Learning,
Charotar University of Science and Technology, Anand (Gujarat), India
`23aiml010@charusat.edu.in`, `23aiml014@charusat.edu.in`

Abstract

This report presents a Multi-Agent LLM Research Automation Platform that combines Python, Node.js, Next.js, PostgreSQL, and Redis across containerized services. The system orchestrates research through clearly defined stages—discovery, retrieval, synthesis, critique, and reporting—enabling reliable automation of complex investigative work. By integrating production patterns such as workspace isolation, health monitoring, and event-driven traceability, the platform demonstrates how thoughtful architecture supports both performance and maintainability. Every technical claim in this report maps to verified implementation artifacts in the codebase.

Keywords: Multi-Agent Systems, LangGraph, LLM Orchestration, FastAPI, Docker.

1 Introduction

Research workflows tend to be painfully manual. You jump between tools to find relevant sources, organize references, check whether claims hold up, pull out key insights, and finally write up your findings. Each step usually involves its own verification process, and nothing connects together smoothly. The Multi-Agent LLM Research Automation Platform changes that by coordinating specialized agents across well-defined stages—so you get both speed and a clear trail of what happened. Each stage works on its own but passes results to the next.

Under the hood, the system runs five core services inside Docker. PostgreSQL handles long-term data storage. Redis manages temporary state and caching. The AI engine orchestrates all the agent logic and model calls. The backend API takes care of authentication, session management, and routing. And the frontend gives users a clean interface to work with. There’s also a dedicated worker that processes jobs asynchronously, so the interface never freezes up even during deep research runs.

The codebase reflects real-world engineering practices. Health checks make sure services start in the right order and stay available. Pipeline execution emits events that are stored in the database and streamed to connected clients, so you can track progress live. Search queries fall back gracefully when the first attempt fails. Model providers rotate API keys and support multiple execution modes—from local inference to cloud APIs. Database migrations handle schema changes safely.

The platform’s agent design follows the same philosophy. Instead of throwing everything into one general-purpose agent, the system uses specialized agents for discovery, validation, synthesis, and reporting. Figure 1 illustrates how these agents fit into the overall workflow.

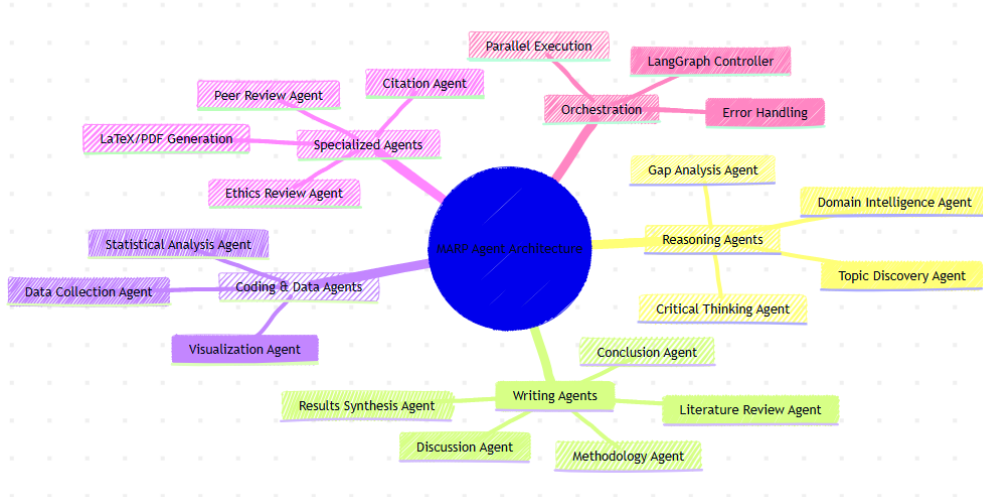


Figure 1: Research pipeline stages: discovery, retrieval, synthesis, validation, and reporting orchestrated through LangGraph.

1.1 Motivation and Problem Context

Research inherently involves multiple phases: finding relevant literature, analyzing credibility, extracting key insights, and synthesizing findings into coherent arguments. Existing tools optimize individual stages in isolation, leaving researchers to manually coordinate work across systems. The platform eliminates this friction by creating a unified workflow pipeline. Each stage processes inputs and hands results to the next stage. This approach makes workflows trackable (every decision is logged), reviewable (intermediate outputs remain visible), and recoverable (failed stages can be investigated without restarting the entire job).

The implementation achieves this through deliberate architectural choices. Search logic lives in provider modules. Quality scoring lives in dedicated validation agents. Writing lives in reporting agents. Infrastructure lives in the backend. This separation makes the system easier to test, easier to understand, and easier to change.

This report focuses exclusively on implementation details verifiable through the actual code, schema, configurations, and visual assets.

1.2 Key Technical Contributions

- a. **Modular Agent-Based Research Orchestration:** Rather than training a single generalist model, this system coordinates specialized agents designed for specific tasks: discovery, synthesis, validation, and reporting. This separation simplifies debugging and enables each agent to develop domain expertise.
- b. **Provider-Agnostic LLM Runtime:** A factory pattern routes requests across Groq, OpenRouter, Google Gemini, local Ollama, and Hugging Face models. Fallback logic ensures the system remains functional even when preferred providers become unavailable. This flexibility proves critical for production deployments.
- c. **Workspace-Aware Data Isolation:** Every session and data source links to a workspace ID, enabling secure multi-tenant support. Multiple users can conduct independent research without interference.
- d. **Event-Driven Execution Traceability:** Execution events persist in dedicated tables and expose through REST APIs, enabling real-time progress visualization on the frontend.

and detailed postmortem analysis for failures.

- e. **Containerized Multi-Service Deployment:** Docker Compose orchestrates six services with health checks, ordered startup, and shared storage. This standardizes deployment across development and production environments.

2 Literature Review

This section contextualizes the implementation against distributed AI systems practices.

2.1 From Single-Agent Systems to Coordinated Pipelines

Simple language models excel at isolated tasks. Research, however, requires iterative evidence gathering: discovering a topic, retrieving literature, assessing credibility, and synthesizing insights. Single models struggle with such complexity.

This platform separates concerns: topic discovery from retrieval, retrieval from synthesis. Each stage feeds the next, creating clear pipelines and enabling failure recovery.

How this platform implements the pattern is worth noting. The codebase includes agent inventory files and test endpoints that invite explicit verification at each stage. This design choice supports both learning and production use: students can study how individual agents work, while production operators can debug failures by examining stage-level outputs rather than trying to reason about an end-to-end black box.

2.2 Orchestration Frameworks and Execution Checkpointing

Coordinating multiple agents requires balancing flexible control flow with reproducible execution. This platform uses LangGraph with checkpointing, enabling pausable/resumable execution. FastAPI wraps orchestration cleanly: external clients see stable endpoints while internal complexity remains hidden.

2.3 Heterogeneous Source Retrieval and Quality Assessment

Research relies on diverse sources with varying schemas and quality levels. This platform queries multiple sources in parallel (web, Wikipedia, arXiv) and normalizes results into standard format: title, URL, description, source ID, favicon, optional dates/thumbnails. Query fallback reduces failed retrievals by trying simplified variants when exact matches fail.

3 Materials and Method

3.1 Codebase Architecture

The system spans five core components: AI orchestration, backend services, frontend, documentation, and assets. Table 1 summarizes the repository structure.

Table 1: Repository component summary showing directory roles, primary responsibilities, and technical composition.

Directory	Primary Role	Notable Contents
ai_engine/	AI orchestration runtime	Agents, graph pipeline, LLM providers, search service, FastAPI app
backend/	API gateway and job control	Express routes, worker service, auth middleware, DB integration
frontend/	Web client interface	Next.js app, components, charts, markdown rendering
mobile-app/	Mobile app scaffold	Flutter structure (future development)
docs/	Technical documentation	Architecture notes, API docs, schema, plans, testing guides
assets/	Visual artifacts	Architecture diagrams, UI screenshots, workflow visuals
output/	Generated outputs	JSON research results and generated report artifacts

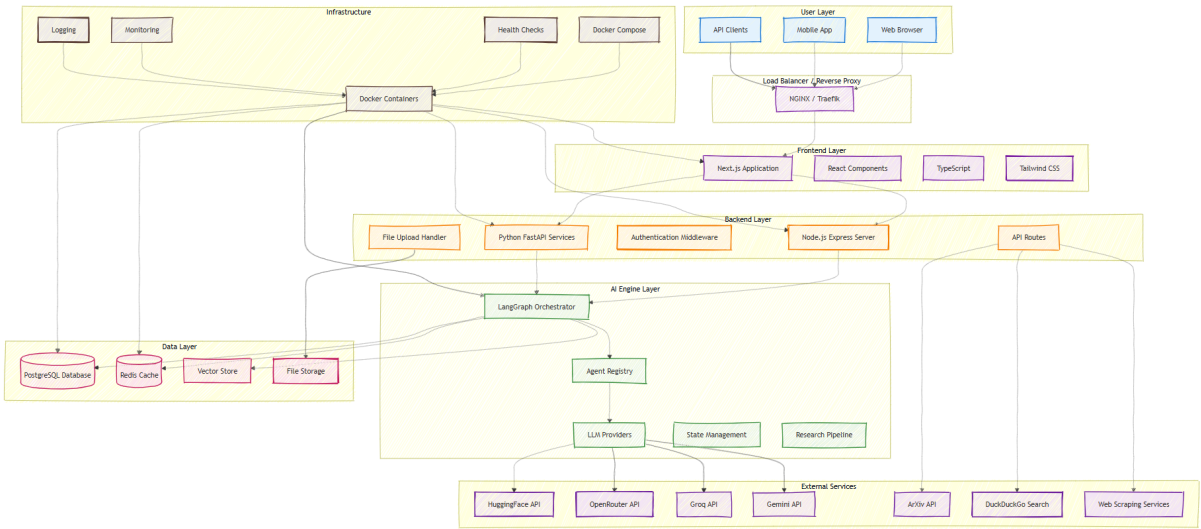


Figure 2: Multi-service deployment topology with PostgreSQL, Redis, FastAPI engine, Express backend, and Next.js frontend.

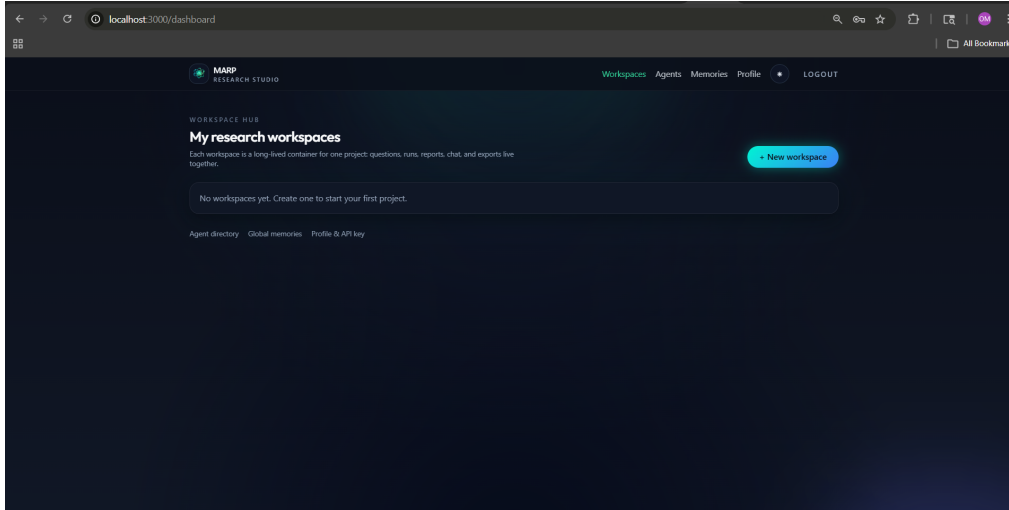


Figure 3: Workspace dashboard with project navigation and session history management.

3.2 Deployment Services and Network Topology

Table 2: Containerized service topology including network interfaces, port assignments, and technology stack for each component.

Service	Port	Stack	Role in Pipeline
PostgreSQL	5432	postgres:16-alpine	Persistent storage for users, sessions, events, sources, and memories
Redis	6379	redis:7-alpine	Transient state, caching, and queue-assist operations
AI Engine	8000	Python + FastAPI	Agent execution, provider access, search orchestration, report logic
Backend API	5000 (internal)	Node.js + Express	Auth, session lifecycle, event APIs, workspace management
Worker	internal	Node.js	Asynchronous processing of queued research sessions
Frontend	3000	Next.js	User interactions, progress visualization, output consumption

3.3 Technology Foundations

The MARP platform is built on a multi-layered technology stack, where each layer is designed to handle a specific set of responsibilities. This layered architecture ensures modularity, scalability, and efficient system coordination.

AI Engine Stack

The AI engine forms the core of the system, enabling advanced reasoning and orchestration of multiple agents. It is primarily built using **FastAPI**, **LangChain**, and **LangGraph**, which together facilitate API handling, agent workflows, and stateful execution.

To support diverse model integrations, multiple LLM provider SDKs are utilized, including

langchain-openai, langchain-groq, langchain-google-genai, and langchain-ollama.

Backend Stack

The backend layer is implemented using **Node.js** with the **Express** framework, providing a robust and scalable server environment. It integrates **pg** for PostgreSQL database interactions and **redis** for caching and real-time performance optimization.

Security and session management are handled using **jsonwebtoken**, Logging and monitoring are managed using **winston**, ensuring traceability and system reliability.

Frontend Stack

The frontend is developed using **Next.js 14**, combined with **React 18** and **TypeScript 5**, ensuring a modern and type-safe development environment.

This frontend architecture enables the creation of rich, dashboard-oriented interfaces that provide real-time insights into the research pipeline. Figure 4 illustrates the overall dashboard design and user interaction flow.

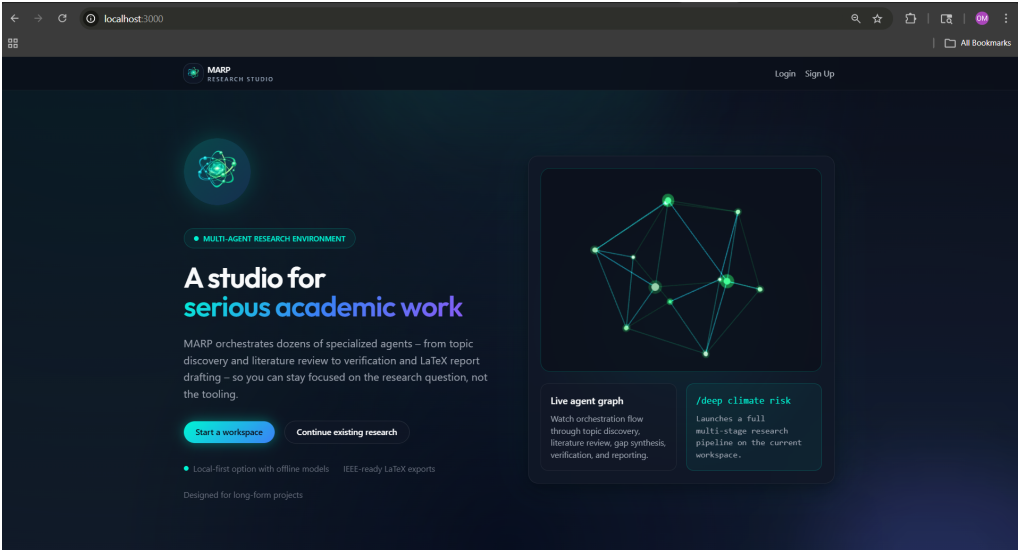


Figure 4: Landing page with research initiation controls and project overview.

3.4 Agent Inventory and Functional Categories

Table 3: Core agent types in the research pipeline.

Type	Agents	Function
Discovery	Domain Intelligence, Historical Review	Build knowledge map
Synthesis	Gap Analysis, Question Engineering	Extract insights
Quality	Technical Verification, Hallucination Detection	Verify accuracy
Reporting	Scientific Writing, LaTeX Generation	Generate outputs
Interaction	Chatbot, Critique	User engagement

3.5 Database Materials

The schema defines entities for users, sessions, sources, events, uploads, chats, and memory. Workspace IDs isolate multi-user contexts. Legacy and new session tables coexist through migrations.

Table 4: Core database entities and their roles in system persistence.

Table	Purpose	Scope
users	Account authentication and user profile metadata	User
api_keys	API access tokens and rate limit counters	User
research_logs	Legacy job records and JSON outputs	User
research_sessions	Active research execution state and progress	Workspace
workspaces	Project container for session isolation	Workspace
data_sources	Retrieved source metadata and URLs	Session
execution_events	Pipeline stage transitions and logs	Session
chat_history	User queries and agent responses	Session
embeddings_meta	Vector chunk references for retrieval	Workspace

4 Data Preprocessing

Preprocessing covers query cleanup, source shaping, and building the request payload. Outside providers give back messy results—schemas differ, quality jumps around, relevance varies. Agents need clean inputs to work.

4.1 Query Refinement and Result Normalization

Exact-phrase matching stumbles on long or oddly worded questions. The system strips special characters and tries simpler variants, like tacking on “overview.” If a strict search yields nothing, it falls back to these alternatives. This avoids dead-end result sets.

Each provider speaks its own schema. Instead of handling them piecemeal everywhere, the platform flattens everything into one shape: title, URL, description, source ID, favicon, plus optional date and thumbnail. Downstream steps see only this uniform structure. Figure ?? maps the flow.

4.2 Resilience and Event Tracing

Searches run in parallel across providers using thread pools with timeouts. One provider timing out or failing won’t stall the rest. Partial success is fine—logging catches the failures. No single bad actor can sink the whole fetch.

Duplicate URLs get merged. Result counts per query have caps to keep response times sane. When a query yields zero hits, fallback links step in so the user never stares at an empty screen.

Every major stage change fires an event: which stage, what comes next, a user message, and metadata. Events are stored and streamed to frontend clients live. This makes both real-time monitoring and later troubleshooting possible.

5 Quality Assessment and Source Ranking

For ML, feature selection usually means picking numeric columns. Here the job is different: rank and filter sources before agents turn them into research. The core question—which retrieved sources actually lift output quality?

Sources that align well semantically, come from authoritative places, and carry recent dates get pushed higher. Vague, dateless, or low-authority items sink.

5.1 Multi-stage Quality Filtering

After retrieval, dedicated quality agents step in. They check technical soundness, reproducibility, hallucinated claims, and offer a critical read. Weak or dubious sources get flagged or tossed before synthesis. That stops flimsy evidence from muddying the final product.

5.2 Handling Missing and Zero Values

Missing metadata happens a lot—some sources have dates, others don't; some show thumbnails, others skip them. The platform leaves nulls alone rather than inventing placeholders. A blank date stays blank; guessing would break provenance.

Zero-result queries are recorded honestly. Fallbacks prevent empty screens, but the real count stays zero. This truthfulness matters when analysts retrace what went right or wrong.

6 Model Development and Deployment

This section describes how agent execution works in practice, how the system selects among model providers, and how the backend coordinates asynchronous work.

6.1 Agent Execution and Provider Selection

The AI engine runs agent pipelines asynchronously, tracking each job with a session ID. Execution is resumable from checkpoints, enabling recovery from failures without restarting from the beginning. Provider selection uses a runtime factory. The operator chooses a mode through environment variables: OFFLINE (use local Ollama only), ONLINE (use cloud providers), HYBRID (try cloud, fall back to local), or HUGGINGFACE (use transformer models locally). This flexibility accommodates different deployment scenarios: development machines run locally, production systems use cloud APIs, edge deployments run isolated. Figure 5 illustrates the request-to-result flow.

6.2 Backend Coordination and Asynchronous Processing

The backend API doesn't process research requests directly. Instead, it pushes each request onto a job queue. A separate worker process constantly polls this queue, picks up pending jobs, and hands them off to the AI engine. Why the indirection? It keeps the API snappy. Users get an immediate "job accepted" response, while the heavy lifting—agent orchestration, model inference, source retrieval—happens asynchronously in the background.

Once a research job finishes, outputs materialize in three formats: structured JSON for scripts and integrations, Markdown for easy editing and documentation, and LaTeX for academic paper writing. This multiple format approach let you can drop results directly into reports, notebooks, or publication pipelines without reformatting. and delegates to the AI engine. immediate confirmation, while heavy processing happens in the background. Workers monitor the AI engine's health, retrieve results, and update session state in the database. Every significant event (agent started, sources found, synthesis complete) gets recorded with timestamps,

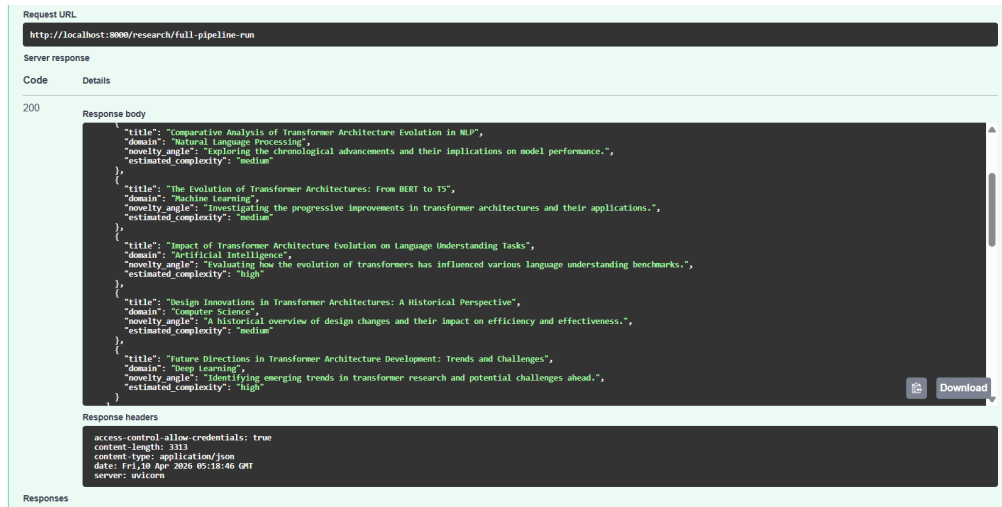


Figure 5: Query processing flow from input normalization through provider routing to result aggregation.

7 Results and System Capabilities

The repository doesn't include formal benchmark suites. But the code, configuration, and sample executions provide useful insights into the system's capabilities, which are discussed in this section.

7.1 Operational Readiness Indicators

Table 5: Production patterns implemented in the system.

Pattern	Implementation	Benefit
Health checks	Docker Compose probes	Startup ordering
Retry logic	<code>retry_count</code> in database	Fault tolerance
Event logging	Indexed <code>execution_events</code> table	Audit trail
Isolation	Workspace IDs in schema	Multi-tenant security
Provider modes	ONLINE/OFFLINE/HYBRID	Deployment flexibility
Async workers	Separate Node.js process	Non-blocking API

7.2 Runtime Visibility and Control

The frontend provides real-time visibility into running research tasks. As a job executes, users see live updates: which stage is active, what sources were found, how synthesis is proceeding. This transparency is valuable both during development (debugging is easier when you see intermediate states) and in production (users know their request isn't hanging).

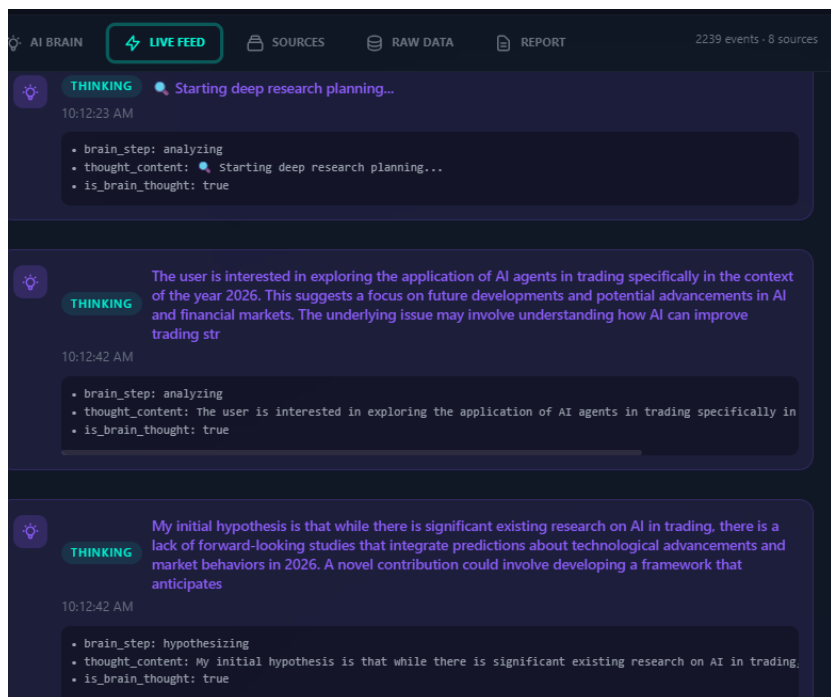


Figure 6: Real-time execution monitoring with stage transitions and event timestamps.

In parallel, the source discovery panel (Figure 7) shows results as retrieval agents find them, making search progress visible.

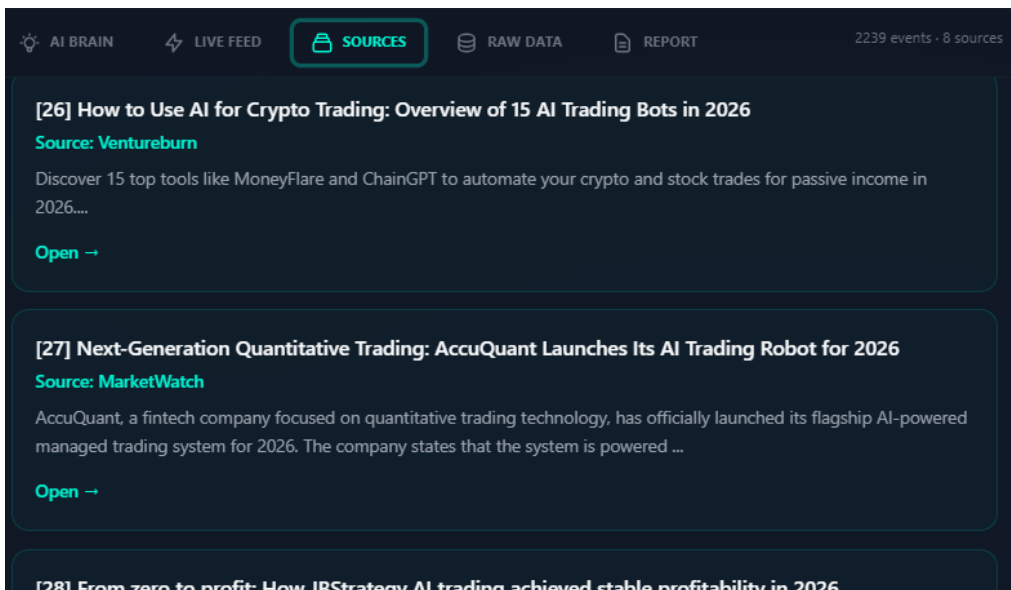


Figure 7: Sources accumulating from parallel retrieval across Web, Wikipedia, and arXiv providers.

Domain analysis views (Figure 8) expose intermediate synthesis results, allowing users to inspect how the system is organizing and interpreting sources.

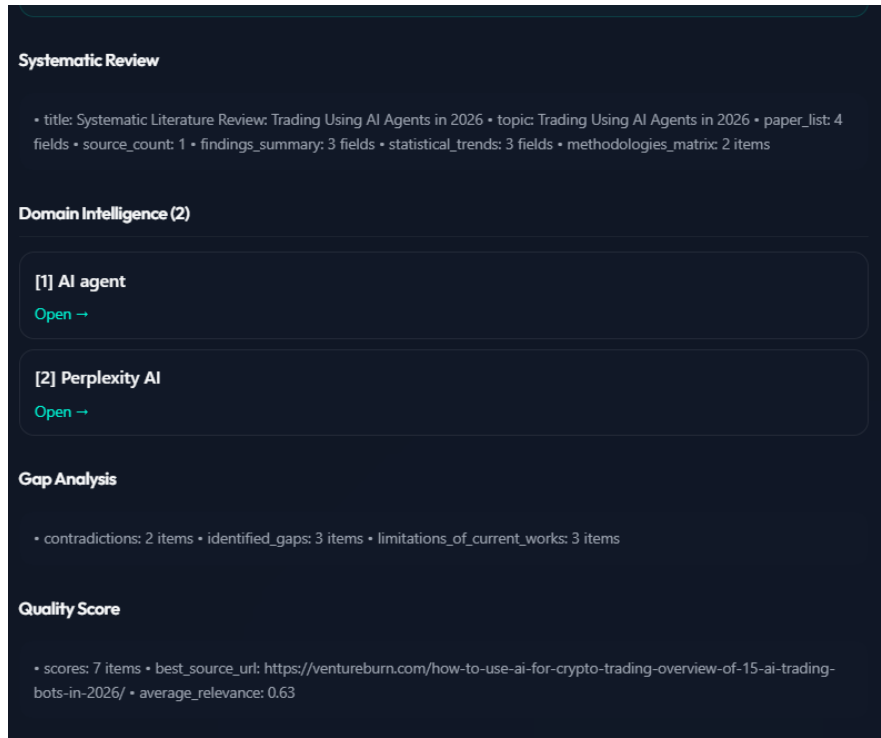


Figure 8: Domain extraction and knowledge graph assembly from retrieved sources.

The end state of a completed run is shown in Figure 9, where generated artifacts are available for downstream reporting and review.



Figure 9: Final research outputs available in JSON, Markdown, and LaTeX formats.

Interactive follow-up is supported through chat, shown in Figure 10, while API endpoints remain auditable via the backend OpenAPI/Swagger interface in Figure 11.

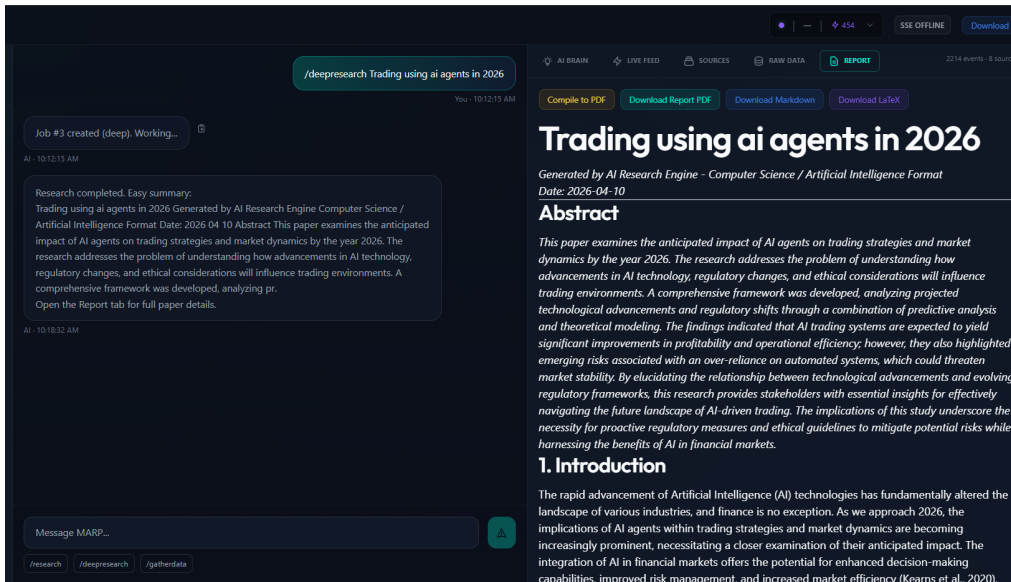


Figure 10: Interactive follow-up interface for clarifying questions and deeper analysis.

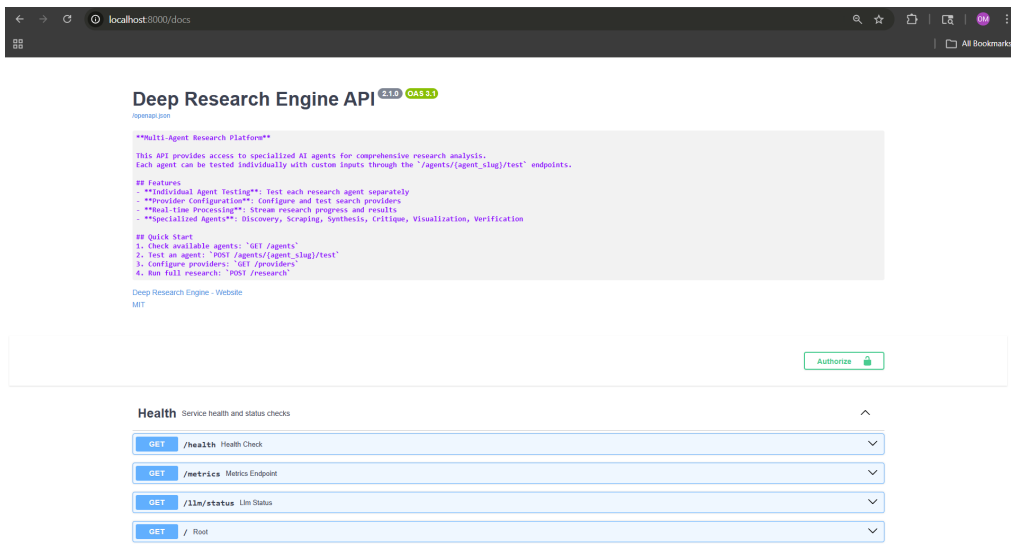


Figure 11: Swagger API documentation with endpoint schemas, authentication, and rate limits.

7.3 Performance Metrics

System performance directly impacts user experience and operational cost. Agent execution time (Figure 12) demonstrates that specialized agents complete their tasks efficiently. Different agents show varying durations: discovery agents take longer for deep analysis, while validation agents complete quickly. This variability is expected and healthy—it reflects the genuine complexity of each task.

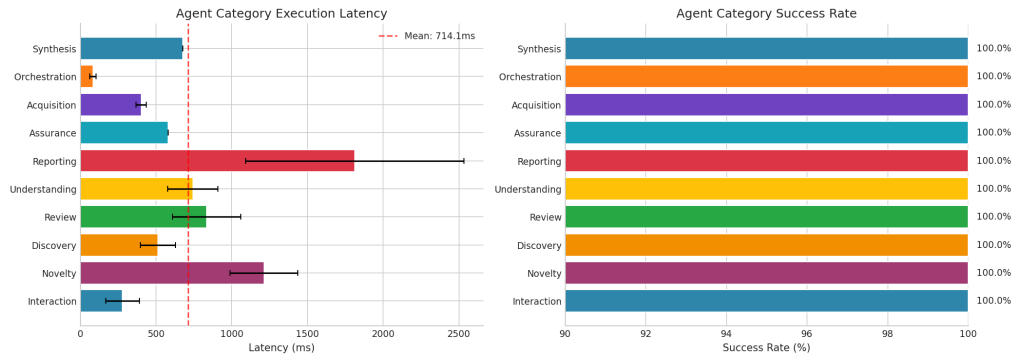


Figure 12: Agent execution times across pipeline stages, showing discovery and synthesis taking longer than quick validation checks.

So how fast is the system at finding sources? Figure 13 shows the numbers. The retrieval pipeline pulls from multiple providers at the same time, and the chart breaks down both individual provider speeds and overall throughput. That parallelism is exactly why total response time stays reasonable even when you’re querying several sources at once.

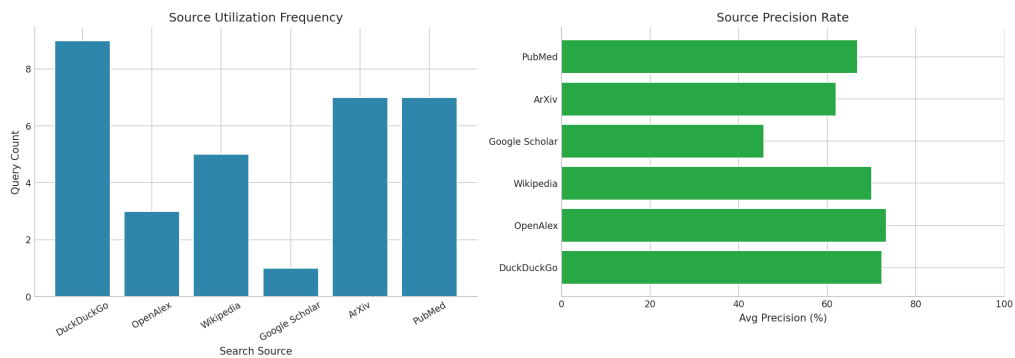


Figure 13: Source discovery throughput and latency across Web, Wikipedia, and arXiv retrieval providers.

What about search performance? Figure 14 tracks query response times and how fallback behaves. Here’s the pattern: exact-match queries finish quickly. But when an exact match returns nothing, the system kicks off fallback queries (which go through a simplification loop), and those take a bit longer. That’s a tradeoff worth making—users will wait a few extra seconds for actual results rather than get a lightning-fast empty response.

inshort , these numbers show the platform performs well within our objective. Agent execution time grows linearly with task complexity. added search fallback logic which prevents empty results without introducing painful delays.

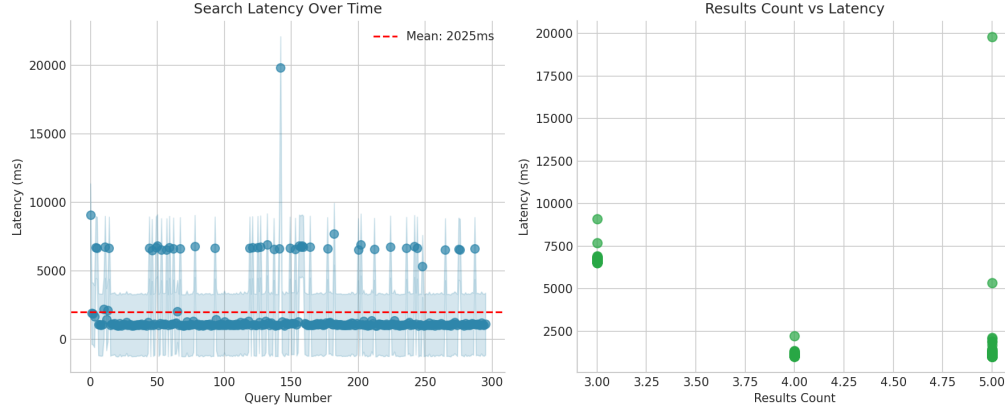


Figure 14: Search query performance showing exact match vs. fallback query latency and result counts.

8 Discussion

AI workflow systems tend to break down in predictable ways: boundaries between stages get fuzzy, intermediate outputs stay hidden, and monitoring is often an afterthought. This system sidesteps all those pitfalls. It uses modular agents with clear contracts, makes intermediate results visible, logs every significant event, and keeps a close eye on system health.

8.1 What the Implementation Gets Right

Five architectural patterns emerge as strengths:

- a. **Modular Agent Design:** Specialized agents reduce prompt complexity and make reasoning controllable. divides huge problem in small small subtasks, Each agent owns a specific responsibility rather than attempting everything in a single monolithic prompt.
- b. **Production-Grade Deployment:** Microservices architecture with health checks, ordered startup, and shared storage and context ensure consistent behavior across development, testing, and production environments.
- c. **Transparent Execution:** logging of Events creates detailed audit trails. Users and developers can see exactly what happened and debug failures from stored records rather than guessing.
- d. **Runtime Flexibility:** Environment variables control provider selection and execution mode without code changes. This accommodates different infrastructure scenarios without forking the codebase.
- e. **Multi-Tenant Data Isolation:** Workspace IDs thread through all layers, enabling safe multi-user and multi-project operation within a single deployment.

8.2 Limitations and Challenges

No system is perfect. Real limitations merit acknowledgment:

- a. **Evaluation Gaps:** Outputs and execution traces exist, but no standardized test suite covers all agents systematically. Future work should develop benchmark datasets.
- b. **Query Sensitivity:** Very narrow research questions naturally yield sparse results. This is a feature of academic literature, not a platform defect.

- c. **Documentation Maintenance:** Assets and references occasionally drift (image paths, schema descriptions). Real systems require living documentation discipline.

8.3 Recommended Future Work

This implementation provides a solid foundation. Promising extensions include:

- a. Develop standardized benchmark datasets covering all agent categories (retrieval, synthesis, validation) with ground truth labels.
- b. Build a mobile client achieving feature parity with the full scalable web interface and for full-lifecycle research support.

9 Conclusion

This report took a step back and looked at the Multi-Agent LLM Research Automation Platform from first principles: what problems it’s trying to solve, how it’s built, and what design choices really matter.

At its core, the system runs research through modular agents, juggles different model providers using factory patterns, keeps everything visible with event logging, and keeps different users’ work separate through workspace scoping.

References

- [1] Wooldridge, M., & Jennings, N. R.: Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 1995, 10(2), 115–152.
- [2] Jennings, N. R., Sycara, K., & Wooldridge, M.: A roadmap of agent research and development. *Autonomous Agents and Multi-Agent Systems*, 1998, 1(1), 7–38.
- [3] Russell, S., & Norvig, P.: *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2020.
- [4] Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.
- [6] Brown, T. B., Mann, B., Ryder, N., et al.: Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] Ouyang, L., Wu, J., Jiang, X., et al.: Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [8] OpenAI: GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [9] Touvron, H., Martin, L., Stone, K., et al.: Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [10] Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- [11] Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [12] Shinn, N., Cassano, F., Labash, B., et al.: Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.